Legacy Banking Modernisation Platform

Proving the rewrite behaves exactly like the original

Team StudyEdge · Odisha University of Technology and Research, Bhubaneswar
Digital Nurture Hackathon 2026 · Theme: Banking and Finance Technology

---

The one-line thesis

Legacy modernisation fails on verification, not translation. We make behavioural equivalence something a bank can actually prove.

Part 1 — The problem

The situation

Indian banks, insurers and public-sector institutions run their core operations on COBOL programs written in the 1980s. Everyone involved knows this is a liability. Everyone wants out. Almost nobody has managed it.

Take one program — the engine that calculates interest on savings accounts. Four thousand lines. Written in 1987. Modified roughly two hundred times since.

Every one of those modifications was a response to something real: a regulatory change, a leap-year bug, a customer complaint about rounding on the third decimal, a fix for accounts opened on the 31st of a month.

None of it was documented. The engineers who made those changes have retired.

The misdiagnosis

The common assumption is that modernisation is hard because translating COBOL into Java is hard.

It isn't. Commercial transpilers have existed for decades. Language models do the job reasonably well. Refactoring and code-generation platforms are a mature product category.

Translation is the solved half. It was never the half anyone was stuck on.

The real problem

Ask the obvious question: where is the document that states what this program is supposed to do?

There isn't one. It was never written. The rules live only inside the code itself.

The code is the specification.

The only ways to learn a business rule are to read four thousand undocumented lines, to run the program and observe what it does, or to ask someone who retired in 2009.

Why nobody risks it

The bank rewrites the program. The new version compiles. It passes the tests the team wrote. Should they deploy it?

They cannot answer that question. Nobody knows what the old program does, so nobody can confirm the new one does the same thing.

And the tolerance for error is zero:

Failure mode	Consequence
Rounding drift	Silent, compounding, discovered at audit
Date boundary handling	Loans mature on the wrong day
Threshold logic	Wrong rate applied to an entire tier

A discrepancy of one paisa across millions of accounts is a reportable regulatory event, not a bug report.

So the institution keeps running software it cannot safely change. That is not cowardice — it is the correct decision given what can and cannot be proven. Multiply it across thousands of institutions and the modernisation backlog is measured in decades rather than quarters.

The bottleneck is verification, not code generation.

Part 2 — The solution

Why the obvious approach fails

The intuitive move is to point a language model at the COBOL and ask it to explain the business rules. The output looks excellent — clean, confident, well organised prose.

And some percentage of it is wrong.

That is worse than useless. A confidently stated wrong rule is more dangerous than no rule at all, because an engineer will build on it. If the only output is a model's summary, the system is a liability generator.

Extraction cannot be the product. **Verification has to be the product.**

Architecture

01 — Extract. Parse the legacy source, decompose it into functional units, and state each unit's business rules in plain language. Every rule carries a pointer to the exact lines that produce it. Traceability is what makes review affordable: an engineer confirms a claim by reading three lines instead of four thousand.

02 — Reimplement. Generate the modern equivalent. This is the commodity step and the least interesting part of the system.

03 — Verify. Generate an input corpus targeting boundaries — zero, negative, maximum values, month and year rollovers, leap days, rounding midpoints. Execute both the original program and the reimplementation against every

input. Compare the outputs.

The key idea: the legacy system is its own oracle

Testing normally requires knowing the correct answer in advance. Someone has to specify the expected output.

Here that requirement disappears, because the program being replaced already defines the correct answer — by running. Whatever it outputs *is* the current behaviour, by definition. Ground truth comes free from the thing being replaced.

Every mismatch therefore becomes a concrete, reproducible finding rather than a confidence score:

> *Given a balance of 9,999.995 on a leap-year accrual, the legacy program returns 1042.50 and the reimplementation returns 1042.51.*

Not a warning. Not a probability. A specific failing case an engineer can act on in a minute.

The reciprocal loop

Extraction and verification strengthen each other.

If the extracted rules state that balances below ₹10,000 earn 3.5% and above earn 4%, the input generator now knows ₹10,000 is a boundary and tests 9,999 / 10,000 / 10,001.

The comprehension layer makes the testing sharper. The testing layer catches the comprehension layer lying. Neither works alone — and that reciprocity is what separates this from "a language model reads your code."

Honesty as a feature

- **Verified** — confirmed by execution. The input corpus exercised this rule and both programs agreed on every case.
- **Unproven** — extracted but never exercised. Flagged explicitly rather than silently trusted, so the gap is visible to the reviewer.

Coverage is reported rather than assumed.

Part 3 — Scope and feasibility

In scope

Dimension	Decision
Language	COBOL, compiled and executed via GnuCOBOL
Domain	Retail banking interest accrual
Program shape	Pure computation — input in, output out

GnuCOBOL is free, open source, and runs on Linux. The harness compiles the COBOL, executes it, captures the output; runs the modern implementation, captures its output; diffs the two. Real execution on both sides, on a laptop. Nothing is hand-waved.

Explicitly out of scope

- Whole-system migration
- CICS screens and JCL orchestration
- Database-coupled programs
- Non-deterministic logic — system clocks, randomness, external state

Differential testing breaks on non-determinism, so those programs are excluded by design rather than by omission. Narrow scope is a design decision, not a limitation we hope nobody notices.

Known limitation

Input generation quality caps everything. Naive random inputs will not find a rounding bug. The substantive engineering is in generating inputs that hit the boundaries the extracted rules imply — which is precisely the reciprocal loop described above, and where the build effort should go.

Part 4 — Outcome

The system produces three artifacts:

1. **A specification** — readable, traceable documentation for a system that never had any.
2. **A regression suite** — thousands of input–output pairs, retained permanently by the institution.
3. **An equivalence claim** — evidence-backed, and the artifact someone can actually sign.

The first two hold their value even if the migration is never approved. The bank ends up with documentation and tests for a system it previously could not touch.

The pitch in one sentence

Everyone else's submission assumes their AI is right. Ours assumes it is wrong, and proves the answer anyway.